

Programming Assignment #1

A Robot Controller for Line Following (and more!)

Nik Swoboda

nswoboda@fi.upm.es

Robótica y Percepción Computacional
Universidad Politécnica de Madrid

March 3, 2026

The code base for this assignment was included in the virtual machine (it is also available in Moodle: `practicalRobotica-v4.7.zip`).

Included in the zip:

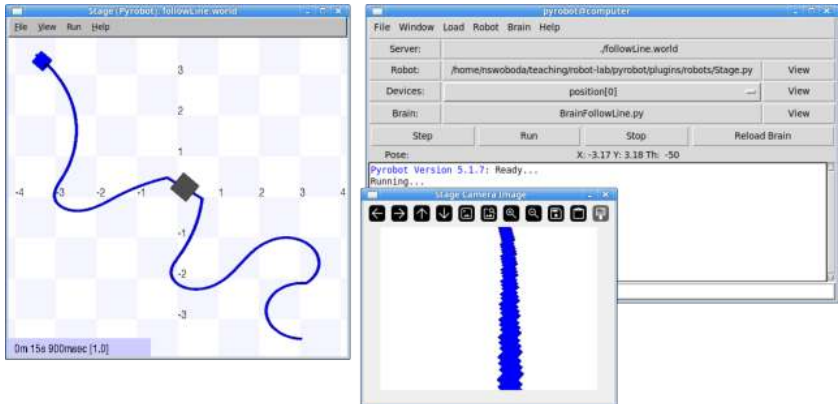
- `BrainFollowLine.py` - A Pyrobot Brain with a basic on-off controller which can more or less follow a simple line.
- `followLine.world` - A Stage world file which loads one of the line images and optionally can contain obstacles.
- `lineBackground-1.png`, `lineBackground-2.png` - Sample line images for use with the Stage world.
- `runFollowLineStageSim` - one line “script” (unix) to run the code

Running the code base

Using the virtual machine and from inside the
practical1Robotica-v4.7 directory, executing

`./runFollowLineStageSim`

and running the brain will give something similar to this:



Pyrobot includes a very basic image processing routine to return the robot's deviation from a line (the error). It works as follows:

- Starting with a gray scale image, discard all but the center-most rows of pixels (by default 4 rows)
- Apply a threshold to separate the line from the background (by default values < 50)
- Calculate the center of mass of all of the line pixels and return:
 - 0 if the line is *centered*
 - 1 if the line is on the *far right edge* of the image
 - -1 if the line is on the *far left edge* of the image
 - Intermediate values proportional to the line's position.

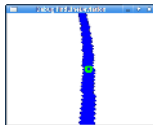
Basic video processing

The following code:

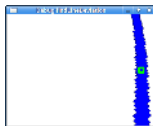
```
foundLine,error = findLineDeviation(imageGray)  
print(foundLine,error)
```

prints the following in the cases shown on the right:

True 0.1125



True 0.7



False 0



Basic video processing

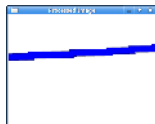
Note: The basic video processing provided by Pyrobot will not:

- detect multiple lines in the field of vision, it will simply return the deviation of the center of mass of all of the lines
- deal well with lines which are close to perpendicular to the robot (it just returns the 'center' of the line which is contained in the center rows)
- detect the angle of the line

True 0.103



False 0



True -0.675



The Assignment

For this assignment you need to modify the `BrainFollowLine.py` file.

You are encouraged to make changes the world file, `followLine.world`. You can change the simulator's background image (to test other lines), move the robot's initial position, include obstacles etc. to thoroughly test your controller.

If you find bugs or have suggestions for improvements, please let us know!

The Assignment

Your basic task is to replace and improve the on-off line-following controller included in the zip.

At the very least we expect submissions which can follow lines like that included in `lineBackground-1.png` and avoid simple obstacles placed on more or less straight portions of the line.

For a better grade, you could consider:

- ability to follow lines with very sharp curves or angles
- ability to recover when losing the line
- more complicated obstacles and/or obstacle placement
- ability to search for a line if not found at the start
(**Note:** The line could be perpendicular to the robot when found! This will not work well with the existing line detection algorithm, but for now you can work around this problem.)
- ...

Changing the Simulation's world

As mentioned previously, please experiment with different lines, robot initial positions, obstacles etc.

To change the background image (in `followLine.world`):

```
bitmap "lineBackground-1.png"
```

To change the initial position of the robot (also in `followLine.world`):

```
# pose = the x, y, z and rotation of the robot
erratic( pose [ -3.3 3.3 0 -40.000 ]
         name "era"
         color "blue")
```

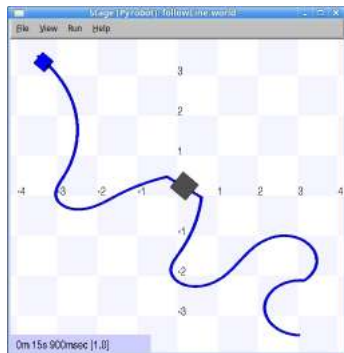
(You can also just drag the robot.)

Changing the Simulation's world

To modify the position and rotation of the obstacle (also in `followLine.world`):

```
box( pose [ 0.17 0.24 0.00  
           -40.11 ]  
      name "box1"  
      color "gray30")
```

You can also change the shape of the box by changing the box model.

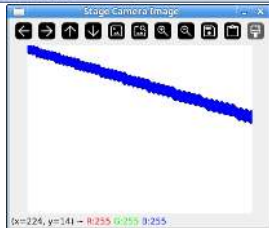
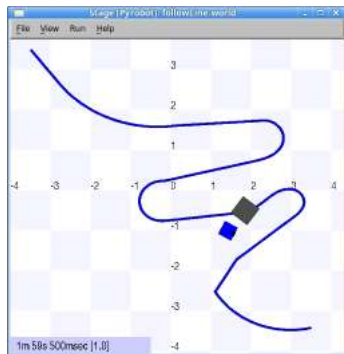


Changing the Simulation's world

Note, some obstacle placements can result in the robot encountering a different part of the line while avoiding the obstacle.

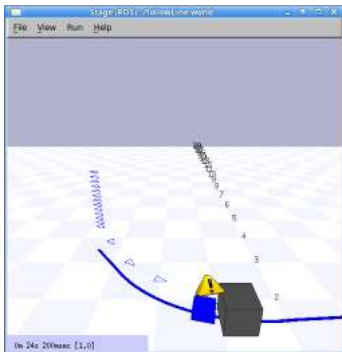
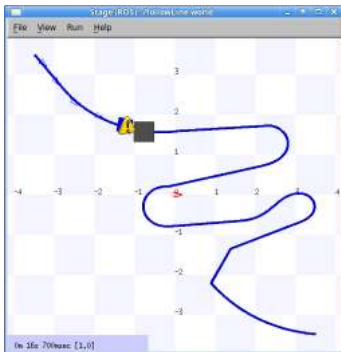
As long as the robot is a reasonable distance from the obstacle, **this will not be considered an error.**

(We will be careful for this not to occur during the exam when using the real robots in the lab.)



Making screenshots for your report

To add a trail you can use the menu option:
View->Trails->Rising Arrows



Some Reminders

- You can work alone or in groups of up to 3 students
- Your work must be submitted in Moodle before **23:55 on March 23th**
- You should submit a zip containing:
 - your Brain and the python files which you modified from the code base (worlds, image files etc.)
 - a brief report (PDF) containing:
 - a description of the changes you made
 - examples of tests you ran showing the capabilities of your code (including screenshots is highly recommended) with a critical analysis of the results
- Only one submission per group is needed, but all the group member's names must be in the code and report.

Good luck!